

AI + 民生赛道

OneCase

把居民反馈整理成事项，跟进处理进度



不理不器 · 个人参赛

一条反馈里，可能有几件事

“三栋二单元的灯又坏了……另外楼下垃圾今天也没人清。”

—— 一条居民留言（合成示例）

- **一条消息** 可能包含多个需要分别处理的问题
- **不同居民** 可能重复反映同一个问题
- **工作人员** 要拆开问题、核对重复，再记入台账

直接用户

社区工作人员
网格员

一个人开发，先做好事项整理

不理不器

个人参赛 · 1 人团队

独立开发

01 产品设计

设计反馈整理流程、状态规则、人工确认和查重方式

02 前后端开发

开发 Web 工作台、模型接口、数据存储与 API

03 测试与文档

单元测试、业务不变量测试、端到端流程测试与产品文档

确认草稿后，再创建正式事项

处理流程示意

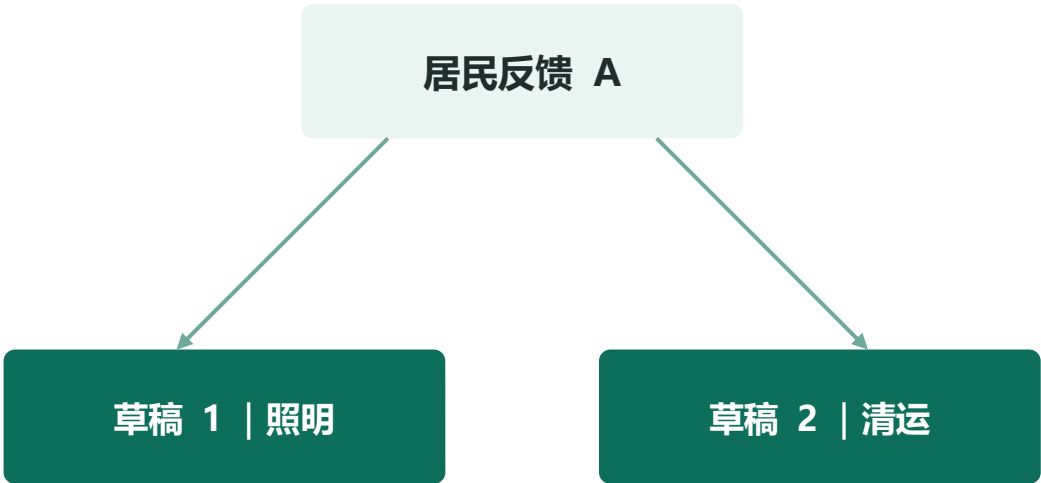


AI 生成草稿，**工作人员检查后决定是否录入。**

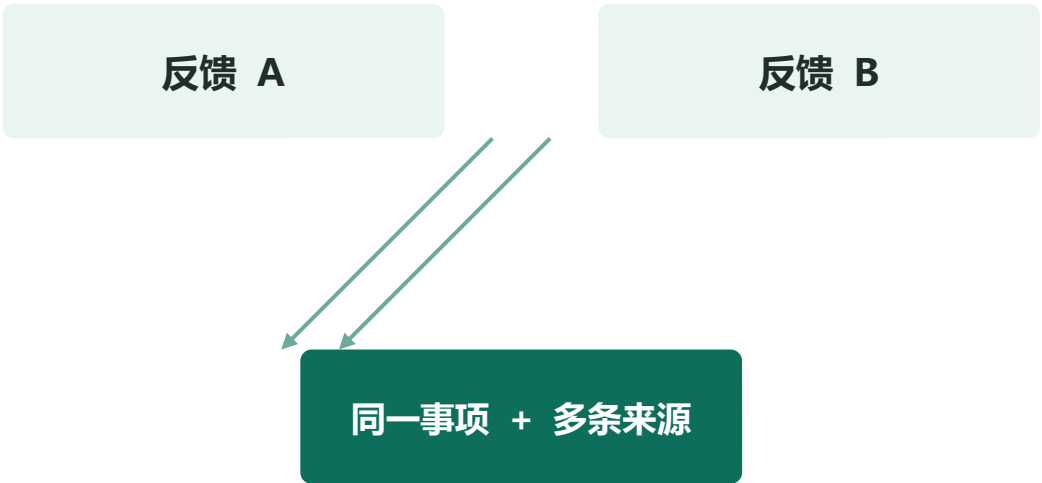
原始反馈、草稿和正式事项分开保存。AI 分析完成后，仍需人工确认。

问题分开处理，来源一起保留

拆开：一条反馈 → 多个事项草稿



关联：多条反馈 → 同一事项（人工确认）



候选按标题词汇、地点、类别和时间排序，并展示依据。地点不同时提醒，由工作人员核对，不自动合并。

现场演示：关联一件，新建一件



演示使用合成数据，现场说明模型模式：真实模型或 Mock。Mock 只用于检查操作流程。

草稿能修改，分析失败可重试

- **草稿可修改**

新建事项采用人工确认后的标题、地点与优先级，并留下修改记录

- **批量确认**

新建、关联和处理记录一起保存；有草稿未处理时不提交

- **防止重复确认**

已确认的反馈不能重复提交；状态只能按规定顺序变更

- **失败后转人工**

AI 分析失败后可重试；符合条件时可手动创建事项

模型整理文字，程序检查规则

Web 工作台

输入、草稿检查、事项跟踪

AI 适配层

接入 Qwen、OpenAI；Mock 用于流程测试

业务控制层

检查字段格式、排列候选、要求人工确认、校验状态变更

数据记录层

保存原始反馈、草稿、事项、来源及处理记录

技术基础 Next.js · React · TypeScript · Zod · Prisma · SQLite

案例与背书：功能与测试

116

项单元测试 全部通过

检查 AI 数据格式、业务规则和查重服务

87

项业务不变量 全部通过

在真实数据库上检查业务规则

22

项端到端流程 全部通过

浏览器流程测试，含 AI 失败后的处理

- **项目成果** 已实现文字整理、人工确认和事项跟踪； AI 失败后可重试或按条件转人工
- **工程证据** 有源码、演示脚本和自动化测试用例，可供检查
- **如实说明** 个人开发项目，尚无可展示的客户或合作数据；实际使用效果还需验证

数据来源：仓库自动化测试套件， 2026-08-31 本机实测通过（另含 6 个子包类型检查全过）

先小范围试用，再讨论收费



使用者

社区工作人员与网格员

潜在付费方

社区服务运营方，预算和采购流程待确认

收费设想

部署与配置服务费 + 持续运维服务费

主要成本

模型调用、部署、安全合规与用户支持

收费方式仍是设想。先确认实际使用价值，目前没有可验证的价格、合同或收入数据。

下一步：测耗时，也检查错误

用同一批经授权、脱敏的反馈，比较人工整理和 OneCase 辅助整理

主要指标

从原始反馈到确认事项的中位耗时

质量指标

漏项、关键字段错误、误关联、人工修改次数

运行指标

模型失败率、转人工次数、单次使用成本

和社区工作人员商定试用标准，再看是否值得继续做。

现在能做什么，试点前还缺什么

当前已实现

- 文字输入与长度校验
- 草稿检查与编辑
- 相似候选与人工确认
- 事项跟踪与统计

试点前补齐

- 身份认证与权限
- 组织数据隔离
- 访问频率限制
- 待确认草稿列表

后续评估

- 图片 / 语音输入
- 微信等渠道接入
- 语义检索（ Embedding ）

图片、语音入口暂不能使用。目前只用合成数据演示，真实试点前还需补齐安全措施。

开发分工与开源组件

主要开源组件 License

组件	License
Next.js / React / Zod	MIT
Prisma	Apache-2.0
Lucide React	ISC

License 依据本机安装包元数据核对；完整依赖清单提交前复核。

开发工作

OneCase 由不理不器独立开发，负责产品设计、业务规则、代码实现和测试。

AI 辅助开发

AI 编程工具用于辅助写代码和测试。需求、方案和最终检查由开发者负责；框架、模型和第三方代码注明来源。

OneCase

AI 先整理，工作人员再确认

保留反馈来源，跟进处理进度

下一步：和社区工作人员一起试用、比较效果